hdoc day 32 by trishil chauhan

Aim

Write a menu-driven C program using separate functions and variable-length 2D arrays to demonstrate that matrix addition is commutative and that matrix multiplication is not commutative in general.

C Program

```
#include <stdio.h>

void input(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void display(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++)
            printf("%5d", a[i][j]);
        printf("\n");
    }
}

int equal(int r, int c, int a[r][c], int b[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

void add(int r, int c, int a[r][c], int b[r][c],
         int sum[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];
}

void multiply(int r1, int c1, int r2, int c2,
              int a[r1][c1], int b[r2][c2],
              int result[r1][c2]) {
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c2; j++) {
            result[i][j] = 0;
            for (int k = 0; k < c1; k++)
                result[i][j] += a[i][k] * b[k][j];
        }
}

void additionDemo(void) {
    int r, c;
    printf("Enter common rows and columns: ");
    if (scanf("%d%d", &r, &c) != 2 || r <= 0 || c <= 0) {
        printf("Invalid dimensions.\n"); return;
    }
    int a[r][c], b[r][c], ab[r][c], ba[r][c];
    printf("Enter matrix A (%d elements):\n", r*c);
    input(r, c, a);
    printf("Enter matrix B (%d elements):\n", r*c);
    input(r, c, b);
    add(r, c, a, b, ab);
    add(r, c, b, a, ba);
```

```
    printf("A + B:\n"); display(r, c, ab);
    printf("B + A:\n"); display(r, c, ba);
    printf("Addition is %scommutative.\n",
           equal(r, c, ab, ba) ? "" : "not ");
}

void multiplicationDemo(void) {
    int n;
    printf("Enter order n for two square matrices: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid order.\n"); return;
    }
    /* Square matrices allow both A*B and B*A. */
    int a[n][n], b[n][n], ab[n][n], ba[n][n];
    printf("Enter matrix A (%d elements):\n", n*n);
    input(n, n, a);
    printf("Enter matrix B (%d elements):\n", n*n);
    input(n, n, b);
    multiply(n, n, n, n, a, b, ab);
    multiply(n, n, n, n, b, a, ba);
    printf("A * B:\n"); display(n, n, ab);
    printf("B * A:\n"); display(n, n, ba);
    if (equal(n, n, ab, ba))
        printf("These matrices commute; try another pair.\n");
    else
        printf("A*B != B*A: multiplication is not "
               "commutative in general.\n");
}

int main(void) {
    int choice;
    do {
        printf("\n1. Prove addition commutative\n");
        printf("2. Test multiplication commutativity\n");
        printf("3. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) break;
        switch (choice) {
            case 1: additionDemo(); break;
            case 2: multiplicationDemo(); break;
            case 3: printf("Exiting...\n"); break;
            default: printf("Invalid choice.\n");
        }
    } while (choice != 3);
    return 0;
}
```

Sample demonstration

For A = [[1, 2], [3, 4]] and B = [[0, 1], [1, 0]]:

```
A + B = B + A =   1   3
                  4   4

A * B =           2   1
                  4   3

B * A =           3   4
                  1   2
```

Result

Addition gives identical results in either order for matrices of the same dimensions. Multiplication is not commutative in general: the example provides a counterexample with A x B different from B x A. Some special pairs can still commute. Square matrices are used in option 2 so both multiplication orders are defined and comparable.